

Attention Backends

Deep Dive into SGLang — Chapter 10

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that turns “a scheduled attention step” into “a kernel can run it.”

- The scheduler has already fixed request rows, positions, and KV slots — the **logical** attention problem.
- A serving system still wants different *execution lanes* for different hardware, model families, and attention shapes.
- The backend’s job: describe that fixed step to kernels **without changing which tokens are visible**.

Goal: by the end you can look at a backend choice and say what it may re-encode, what it must preserve, and how it can fail while still looking plausible.

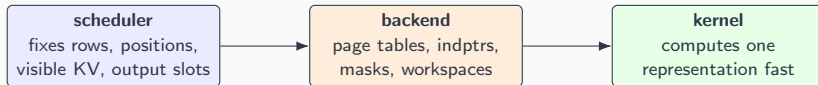
The boundary

Selection and ownership

Metadata and correctness

The boundary

The handoff keeps three layers separate



An attention backend computes one scheduled step. For each query row i the scheduler fixed a visible KV set $\mathcal{V}_i$ and an output row. The backend may re-encode $\mathcal{V}_i$ as page tables, indptr arrays, masks, or graph buffers — and may pick a different kernel family for decode, extend, or MLA.

The Volume I question: does backend metadata still preserve the scheduler's causal view?
Volume II handles the launch.

What a backend must still represent

Every backend choice b^* must keep the output row equal to

$$y_i = \sum_{j \in \mathcal{V}_i} \text{softmax}_j \left(\frac{q_i k_j^\top}{\sqrt{d}} + m_{i,j} \right) v_j,$$

with the **same** request identity, visible positions $\mathcal{V}_i$, mask contribution $m_{i,j}$, and cache-write destination the scheduler fixed.

- Model families (MHA, MQA/GQA, MLA, SWA, NSA) are *fixed inputs*, not choices the backend invents.
- Backend metadata is an *encoding* of $\mathcal{V}_i$ and the output placement — not a license to choose a different history.

Selection and ownership

Three runtime questions

1. **Which backend can run the step?** Server arguments, model family, platform, dtype, page size, and feature gates decide eligibility.
2. **Which metadata view does it build?** One per-step record $M_{b^*}(s)$, prepared once and reused across attention layers.
3. **Does fallback / graph replay / delegation preserve the scheduled view?** One alignment invariant answers all three.

Availability, default choice, and legal use are three separate gates. A name in the catalog does not prove a batch can run.

Definition (Backend compatibility gate)

$G_b(s)$: the *ordered* family of model-family, forward-mode, KV-layout, dtype, graph, workspace, and hardware checks backend b must pass before $M_b(s)$ can be trusted for step s .

- A backend that cannot represent the scheduled histories, page layout, or output mode is **ineligible** — even if it is the fastest kernel for a nearby shape.
- Order matters: SGLang rejects at the first source-visible incompatibility (unsupported dtype, wrong device family).

Ownership transfer, not a file tour

Object owns the decision

... and what stays fixed

server arguments

names & defaults; model family, flags, platform fixed

registry + model runner

the backend object; the selected name fixed

ForwardBatch

the executable step; row order, KV ownership fixed

RadixAttention

the layer call; output rows & cache reads fixed

Ask at each boundary: which object owns the backend decision now, and which parts of the scheduled attention problem are still fixed? A miss at the registry is a *config* error; a leak at the layer call is a *metadata-alignment* bug.

From concept to code: the layer dispatches, it does not decide

RadixAttention.forward routes the call; it never re-decides reuse.

```
from python/sglang/srt/layers/radix_attention.py

if forward_batch.forward_mode.is_extend() and get_forward_context() is not None:
    ...
    unified_attention_with_output(q, k, v, output, save_kv_cache,
                                  self.layer_id, **kwargs)
    return output
else:
    return forward_batch.attn_backend.forward(
        q, k, v, self, forward_batch, save_kv_cache, **kwargs)
```

Two routes, one rule: extend-with-context enters the unified op; otherwise dispatch to `attn_backend.forward`. Neither path changes which tokens a request may attend to.

Metadata and correctness

Many representations, one scheduled batch

Diversity is **representational, not semantic**. All families start from the same ForwardBatch:

Family	Encoding of the same visibility set
Triton	KV indptrs, flat KV indices, qo-indptrs, split counts, mask indptrs
FlashInfer	paged kv_indptr/kv_indices, last-page lengths, wrapper workspaces
FlashAttn	cumulative q/k lengths, cache seqLens, page tables, graph buffers
MLA	latent-cache page indices, query/key-head layout, compressed KV
Hybrid	mode-specific decode/extend metadata, delegated by mode

Compare them by what each must preserve — not by memorizing field names.

The one invariant that catches every variant

Backend-metadata alignment

A backend invocation for b^* is valid only when request identity, forward mode, positions, KV indices, masks, lifetime, batch order, graph shape, and auxiliary state in $M_{b^*}(s)$ describe the *same* scheduled work s .

The dangerous bug keeps shapes *plausible*. For extend row C_{26} :

- **KV-index violation:** right-length segment filled with request B 's slots — the query reads B 's history.
- **Mask violation:** right slots, but the mask leaves C_{28} visible — a future suffix token leaks.

Both preserve tensor shapes while changing the scheduled problem.

Measurement only counts the path that ran

- Backend claims are hardware- and model-sensitive — **measured context is part of the result.**
- Thor/Qwen slice: the decode pair (FlashInfer vs. Triton) is *close* ($\approx 6,37\text{k tok/s}$ each); the extend-heavy pair splits clearly (FlashInfer $\approx 44\text{k}$ vs. Triton $\approx 25\text{k tok/s}$).
- A **backend decision record** per step names the requested mode, selected backend, fallback state, and graph state — before any aggregate is read as backend evidence.

An aggregate row is backend evidence only once the decision record shows which backend actually ran.

What to carry forward

1. Backend selection is a **semantic promise** about the scheduled attention view; kernel details are deferred to Volume II.
2. **Gates** choose a legal family; **ownership transfer** builds the object; **metadata** encodes the visibility set; **the invariant** catches any representation that changes it.
3. Decode, extend, mixed, hybrid, MLA, and graph paths may differ in *representation* — never in identity, positions, KV indices, masks, or output order.
4. Every concept has a **home in the SGLang source**, from `server_args.py` to `base_attn_backend.py`.

Next: Section 1 fixes the boundary — the scheduler hands over a fixed attention problem, and the backend changes lanes without changing meaning.